

Ros Day02 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.10

목차

1. 과제 목표
2. ROS Domain 설정
3. Turtlesim 실행
4. 원 그리기
5. 사각형 그리기
6. 삼각형 그리기
7. 문제점 및 해결

1. 과제 목적

이번 과제에서는 Qt GUI 를 이용하여 turtlesim 을 조종하는 ROS 2 C++ 패키지를 구현하였다. GUI 의 버튼이나 W, A, S, D 키를 입력하면 거북이가 삼각형, 사각형, 원을 그리도록 구성하였다. 각 도형은 서로 다른 색상과 굵기로 출력되며, GUI 에서 현재 cmd_vel 값과 거북이의 위치 및 동작 상태를 확인할 수 있도록 하였다.

2. 패키지 구성

패키지 이름은 parkdoyeon_day2_hw2 이며 ament_cmake 방식으로 생성하였다. 프로그램은 다음과 같이 헤더 파일과 소스 파일을 분리하였다.

turtle_gui.hpp 에는 TurtleGui 클래스와 버튼 슬롯, ROS 2 Publisher, Subscriber, Service Client, 도형 제어에 필요한 변수와 함수가 선언되어 있다.

turtle_gui.cpp 에는 Qt 화면 구성, 버튼과 키보드 입력 처리, ROS 2 통신, 도형별 이동 명령이 구현되어 있다.

main.cpp 에서는 ROS 2 와 QApplication 을 초기화하고 TurtleGui 창을 실행한다.

CMakeLists.txt 에는 rclcpp, geometry_msgs, turtlesim, std_srvs, Qt5 Widgets 의존성과 turtle_gui 실행 파일 생성 설정을 작성하였다.

3. Turtlesim 실행

GUI 는 도형 선택 영역과 ROS 2 상태 출력 영역으로 구성하였다. 도형 선택 영역에는 삼각형, 사각형, 원, 전체 도형, 정지, 화면 지우기 버튼을 배치하였다.

각 버튼에는 다음 기능을 연결하였다.

W 또는 삼각형 버튼을 누르면 빨간색 삼각형을 그린다.

A 또는 사각형 버튼을 누르면 초록색 사각형을 그린다.

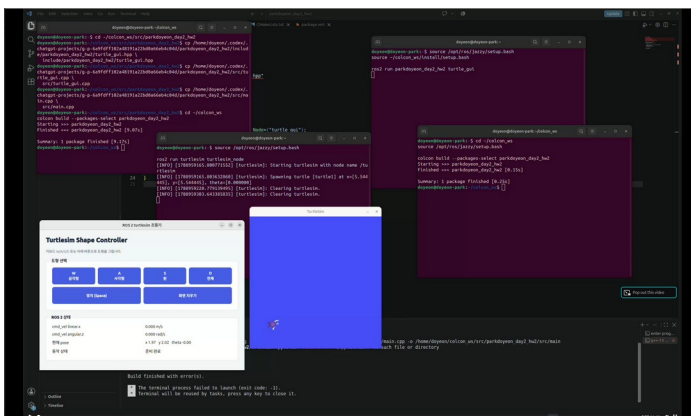
S 또는 원 버튼을 누르면 파란색 원을 그린다.

D 또는 전체 버튼을 누르면 삼각형, 사각형, 원을 순서대로 그린다.

Space 또는 정지 버튼을 누르면 현재 동작을 중단하고 속도를 0 으로 설정한다.

화면 지우기 버튼을 누르면 turtlesim 의 clear 서비스를 호출하여 기존 선을 삭제한다.

QShortcut 을 사용하여 버튼을 직접 누르지 않아도 W, A, S, D 와 Space 키로 같은 기능을 실행할 수 있도록 하였다.



4. ROS 2 통신 구현

거북이의 이동에는 `/turtle1/cmd_vel` 토픽을 사용하였다. `geometry_msgs` 의 `Twist` 메시지에서 `linear.x` 에 선속도를 저장하고 `angular.z` 에 각속도를 저장하여 전송하였다.

`linear.x` 는 거북이가 앞으로 이동하는 속도이며 단위는 `m/s` 이다. `angular.z` 는 거북이가 회전하는 속도이며 단위는 `rad/s` 이다.

GUI 의 ROS 2 상태 영역에는 전송 중인 `linear.x` 와 `angular.z` 값을 소수점 셋째 자리까지 표시하였다. 거북이가 직진할 때는 `linear.x` 값이 증가하고, 회전할 때는 `angular.z` 값이 증가한다. 이동이 완료되거나 정지 버튼을 누르면 두 값이 모두 0 으로 표시된다.

거북이의 현재 위치는 `/turtle1/pose` 토픽을 구독하여 확인하였다. 수신한 `x`, `y`, `theta` 값을 GUI 에 표시하고 이동 거리와 회전 각도를 계산하는 데 사용하였다.

펜의 색상과 굵기는 `/turtle1/set_pen` 서비스를 사용하여 변경하였다. 도형의 시작 위치를 설정할 때는 `/turtle1/teleport_absolute` 서비스를 사용하였다. 화면 지우기 기능에는 `/clear` 서비스를 사용하였다.

5. 도형 그리기 구현

삼각형은 `x 2.0, y 8.0` 위치에서 시작하도록 설정하였다. 펜 색상은 빨간색 RGB 255, 0, 0 이며 굵기는 3 으로 설정하였다. 직선으로 2.5 만큼 이동한 뒤 120 도 회전하는 동작을 세 번 반복하여 삼각형을 완성하였다.

사각형은 `x 2.0, y 2.0` 위치에서 시작하도록 설정하였다. 펜 색상은 초록색 RGB 0, 180, 70 이며 굵기는 5 로 설정하였다. 직선으로 2.5 만큼 이동한 뒤 90 도 회전하는 동작을 네 번 반복하여 사각형을 완성하였다.

원은 `x 8.0, y 4.0` 위치에서 시작하도록 설정하였다. 펜 색상은 파란색 RGB 30, 100, 255 이며 굵기는 7 로 설정하였다. `linear.x` 를 1.2 `m/s`, `angular.z` 를 0.8 `rad/s` 로 동시에 전송하여 원형으로 이동하도록 하였다. 회전한 각도의 합이 360 도에 도달하면 속도를 0 으로 설정하였다.

각 도형의 동작은 명령 큐에 순서대로 저장하였다. 전체 도형 버튼을 누르면 삼각형, 사각형, 원의 명령이 차례대로 큐에 들어가므로 하나의 도형이 끝난 뒤 다음 도형이 실행된다.

6. 문제해결과정

처음에는 도형의 시작 부분이 굵기거나 예상하지 않은 방향으로 선이 그려지는 문제가 발생하였다. 원인은 `teleport` 서비스를 호출한 직후 이전 위치의 `pose` 데이터를 기준으로 이동 거리와 회전 각도를 계산했기 때문이다.

이를 해결하기 위해 `teleport` 후 수신한 `pose` 가 목표 위치와 방향에 도달했는지 확인한 다음 이동 명령을 시작하도록 수정하였다. 목표 위치와 실제 위치의 차이가 일정 범위 안에 들어온 경우에만 `pose` 수신이 완료된 것으로 처리하였다.

또한 도형을 그리는 도중 버튼을 여러 번 누르면 명령이 겹칠 수 있으므로, 현재 도형이 실행 중일 때는 새로운 도형 명령을 시작하지 않도록 하였다. 정지하거나 프로그램을 종료할 때는 `linear.x` 와 `angular.z` 가 모두 0 인 `Twist` 메시지를 전송하도록 하였다.

또한 참고로 원은 파란색으로 정상적으로 출력되지만 `turtlesim` 의 배경도 파란색 계열이기 때문에 다른 도형보다 잘 보이지 않을 수 있다.

7. 실행 결과

패키지를 빌드한 후 turtlesim_node와 turtle_gui를 각각 실행하였다. GUI의 버튼과 키보드 입력을 이용하여 각 기능을 테스트 하였다.

W 입력에서는 빨간색 삼각형이 출력되었다.

A 입력에서는 초록색 사각형이 출력되었다.

S 입력에서는 파란색 원이 출력되었다.

D 입력에서는 세 도형이 서로 다른 위치에 순서대로 출력되었다.

정지 기능을 실행하면 거북이가 멈추고 cmd_vel 값이 0으로 변경되었다.

화면 지우기 기능을 실행하면 기존에 그려진 선이 삭제되었다.

GUI의 cmd_vel, 현재 pose, 동작 상태도 거북이의 움직임에 맞게 변경되는 것을 확인하였다.

